

System Architecture and Technology Stack

1. Overview

Itqan is organised into three parts. A mobile application captures the user's recitation and presents feedback. An application backend manages sessions, comparison, progress and classes. A separate analysis service runs the speech model that converts recitation audio into phonemes and their articulation attributes.

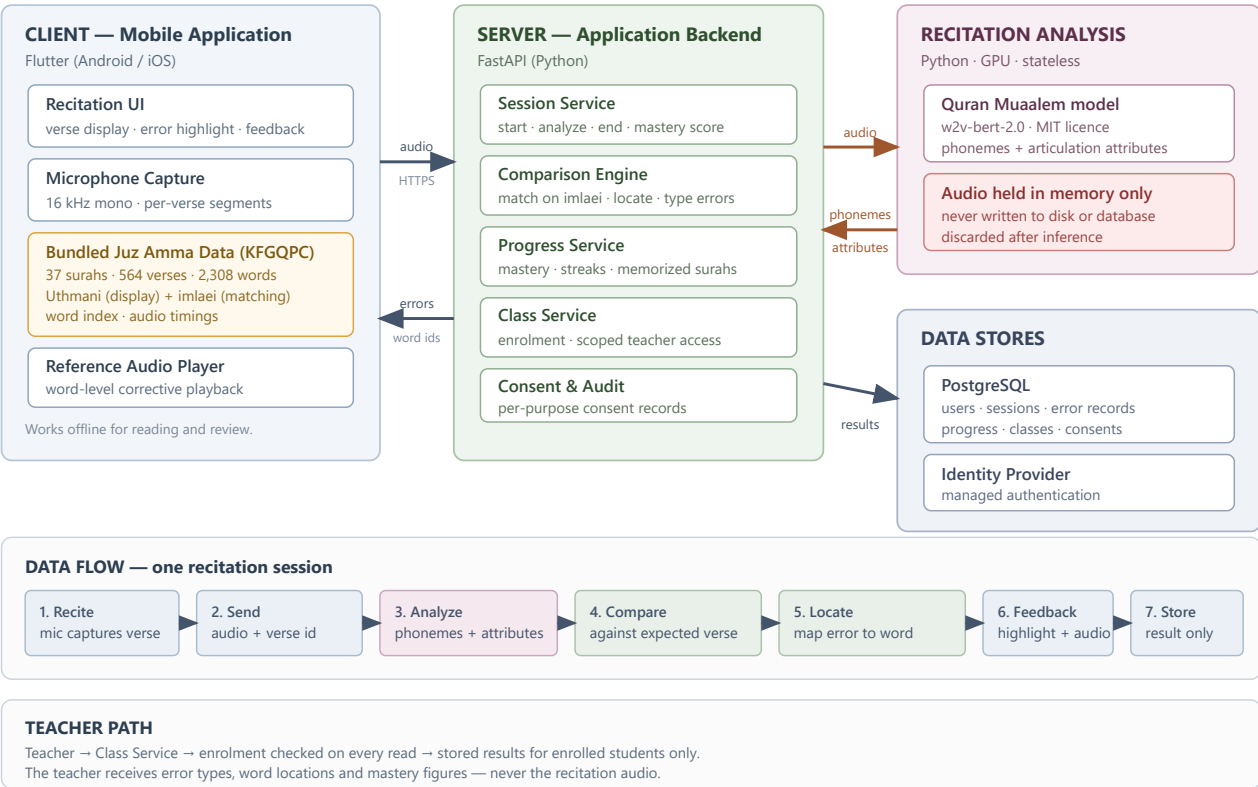
Three decisions shape everything else.

The analysis runs on a server, not on the phone. Running the model on the device would give better privacy and work offline, but the standard tooling for converting this model architecture to run on mobile devices does not support it. Making the project's core feature depend on unproven engineering was judged too great a risk.

The Quranic content is bundled inside the application. Juz Amma is 37 surahs, 564 verses and 2,308 words — small enough to ship with the app. Reading and review therefore work without a network connection, and a demonstration cannot fail because an external service is unavailable.

Recitation audio is never stored. It is held in memory for the duration of the analysis and then discarded. What persists is the result: which word was wrong, what kind of error it was, and the resulting score.

2. Architecture Diagram



Itqan — system architecture. The recitation audio is processed in memory and discarded; only per-word results are stored.

3. Components

3.1 Mobile Application

Built with Flutter, producing both Android and iOS applications from one codebase.

Component	Responsibility
Recitation UI	Displays the verse, highlights words where errors were detected, presents feedback
Microphone Capture	Records the recitation at 16 kHz mono, segmented by verse
Bundled Juz Amma Data	Local database holding the verse text in two forms, the word index, and word-level audio timings
Reference Audio Player	Plays the correct recitation of a specific word, using the bundled timings

Flutter was chosen because the first decision moved the demanding work to the server. What remains on the device is screens, a microphone, a local database and audio playback — work Flutter handles well. Writing separate Android and iOS applications would have doubled the client effort for no benefit the project needs.

3.2 Application Backend

Built with FastAPI in Python.

Component	Responsibility
Session Service	Starts, runs and ends a recitation session; produces the mastery percentage
Comparison Engine	Compares what the model observed against what the verse requires, and classifies each difference
Progress Service	Maintains mastery per surah, average mastery, streaks and memorized surahs
Class Service	Manages enrolment and enforces the scope of a teacher's access on every read
Consent and Audit	Records consent separately for each purpose

The Comparison Engine is where the team's own contribution lies. The speech model reports what it heard; it has no knowledge of which verse was intended. Deciding that a particular difference is a skipped word rather than a mispronounced one, and mapping that judgment back to a specific word on the screen, is logic the team writes.

3.3 Recitation Analysis Service

A stateless Python service holding the Quran Muaalem model (`obadx/muaalem-model-v3_2`, MIT licence, based on `facebook/w2v-bert-2.0`). It accepts audio, returns phonemes with articulation attributes, and retains nothing.

It is deliberately separate from the backend. It is the only component requiring a GPU, so it can be hosted and scaled independently; keeping it stateless means it holds no user data; and isolating it means the rest of the system — reading, progress, class management — continues to work if it is unavailable.

3.4 Data Stores

PostgreSQL holds accounts, sessions, per-word error records, progress, classes and consent records. The data is relational, and enrolment-scoped teacher access is naturally expressed as a join, which makes the access rule easier to enforce correctly than it would be in a document store.

Authentication is delegated to a managed identity provider rather than implemented by the team.

4. Data Flow

4.1 A Recitation Session

1. **Recite.** The user selects a surah and recites. Audio is captured at 16 kHz mono, segmented by verse.
2. **Send.** The application sends the audio and the identifier of the verse being recited over HTTPS. Sending the verse identifier matters: because the expected text is known in advance, the system never performs open-vocabulary recognition. It is not asking *"what did this person say?"* but *"how does what they said differ from this specific verse?"* — a considerably easier question.
3. **Analyze.** The analysis service runs the speech model and returns a sequence of phonemes, each with its articulation attributes. **The audio is discarded at this point** and is never written to disk or database.
4. **Compare.** The Comparison Engine aligns the observed sequence against the expected sequence for that verse. Every mismatch is classified as a wrong word, a skipped word, an added word, a mispronounced letter, or a tajweed error of a named type.

5. **Locate.** Each error is mapped from its position in the phoneme sequence back to a word index in the verse. This is what allows the error to be shown on a specific word rather than somewhere in the verse.
6. **Feedback.** The application receives the error list with word indices. It highlights those words, states the error type, and can play the correct recitation of the affected word from the bundled reference audio.
7. **Store.** Only the result is written: surah, verse, word index, error type, confidence, timestamp and duration. Progress and mastery figures are recalculated.

4.2 What Crosses Each Boundary

From	To	What is sent
Mobile application	Backend	Recitation audio and the verse identifier
Backend	Analysis service	Recitation audio
Analysis service	Backend	Phonemes with articulation attributes
Backend	Mobile application	Error list with word indices, error types, mastery percentage
Backend	Database	Per-word error records, progress, session metadata
Backend	Teacher's application	Error types, word locations and mastery figures, for enrolled students only

Two things never appear in this table, and their absence is deliberate. **Audio never reaches the database** — it exists only in memory, and only for as long as one analysis takes. **Audio never reaches a teacher** — a teacher receives results, not recordings.

4.3 The Teacher Path

A teacher's request reads only what is already stored:

1. The teacher opens a student's progress.
2. The Class Service checks that an **active enrolment** links this teacher to this student at this moment.
3. If it does, stored results are returned — error types, word locations, mastery figures and trends, limited to the enrolment period.
4. If it does not, the request is refused.

The check happens on every read, in the data layer. It is not an interface restriction that could be bypassed by requesting the data directly.

4.4 What the System Stores

Stored	Not stored
Account details and role	Recitation audio
Per-word error records	Speaker embeddings or voiceprints
Progress, mastery, streaks	Location data
Class enrolments and their periods	Any personal data not required by a stated purpose
Consent records, one per purpose	

4.5 Offline Behaviour

Because the Quranic content is bundled, browsing Juz Amma, selecting a surah, reading verses and reviewing stored results all work without a network connection. Recitation analysis, progress synchronisation across devices and any teacher view require a connection.

5. Quranic Content

5.1 Source

The verse text is taken from the **King Fahd Glorious Quran Printing Complex**, through its developer platform at qurancomplex.gov.sa/quran-dev. The Complex describes this platform as a service provided *"to developers, researchers and publishing houses, for use in the content of their applications (desktop, mobile, and browsers) and their research,"* and states that the content is *"trusted and approved by"* the Complex.

This source was chosen for textual authority. For a project submitted at King Saud University, using the authoritative Saudi edition answers the question of who certified the Quranic text, rather than leaving it to be asked.

The Hafs package provides, for each verse, the Uthmani text (`aya_text`), the same verse in ordinary modern orthography (`aya_text_emlaey`), the surah and verse numbers, and the position within the printed mushaf. It does **not** include word-level segmentation — the smallest unit published is the verse — so word boundaries are derived by the team.

Conditions of use. No licence text, attribution requirement, registration step or approval process is published on the developer platform; this was confirmed by searching the full page for the relevant Arabic terms. The Complex's site-wide usage clause reserves rights over its content *"with the exception of certain services, computer files, software tools and others which the Complex authorises for use by the public"* — the developer packages being precisely such files.

The position is therefore that the material is published openly for developers, that mobile applications are named among its intended uses, and that no condition was found attached to it. The absence of stated conditions is not itself a grant of rights, and Itqan credits the Complex as the source of its Quranic text regardless.

5.2 Deriving Word Boundaries

Itqan must identify errors at the level of the individual word, which requires knowing where each word begins and ends both in the text and in the reference recording.

Word-level timing data exists for Quranic recitation, but it indexes words by splitting a **different** edition — the Tanzil Uthmani text — on spaces. If the two editions divided into different words, word number four would refer to different words in each, and the application would highlight or play back the wrong one.

This was tested rather than assumed. Both editions were obtained and compared across all 564 verses of Juz Amma:

- Word boundaries are **identical in every verse**, once one known difference is accounted for.

- That difference is the **Basmalah**. The Tanzil edition includes it as the first four words of the first verse of each surah; the Complex's edition does not. Across 37 surahs this accounts for 148 words — the entire discrepancy.
- The remaining differences are confined to **twenty character-substitution types**, all presentation choices. For example, the Complex's edition uses a font-specific character where the other uses a standard sukun. None affect where one word ends and the next begins.

The available word-timing data can therefore be used with the Complex's text, subject to one rule: remove the Basmalah from the first verse of each surah when mapping to the timing data.

The verification is reproducible. The script `verify_text_alignment.py` performs the comparison and prints its findings, and its output can be included in the report as evidence rather than as an assertion.

Juz Amma	
Surahs	37
Verses	564
Words	2,308

5.3 Matching Against One Spelling, Displaying Another

The Complex's data supplies each verse in two spellings: Uthmani script, and ordinary modern orthography intended for search. This matters because speech systems work considerably better against ordinary orthography — Uthmani script preserves historical spellings that do not correspond straightforwardly to how words sound.

Testing showed that **the two spellings divide into exactly the same words** — 2,308 in each across Juz Amma, with no verse disagreeing. Word number seven refers to the same word in both forms.

The architecture uses this directly: **recitation is matched against the ordinary orthography, while the Uthmani text is what the user sees**. The same word index applies to both, so no mapping layer is needed.

5.4 A Constraint on Display

The Complex states that its smart-devices font *"is not prepared for displaying a complete mushaf page matching the Madinah Mushaf; rather it is used to display the Quranic text at the level of verses only."*

Itqan therefore presents verses individually rather than reproducing a mushaf page layout. This also suits the interaction: the user recites one verse at a time, and the interface highlights words within the verse being recited. A page-faithful mushaf view would require a different package from the Complex.

5.5 Reference Audio

The application bundles a reciter's recording of Juz Amma together with the word-level timings that allow a single word to be played back.

This imposes one constraint on the choice of recording: it must be **the recording the timings were produced from**. Timings are specific to a particular recitation, and applying them to a different recording

would play back the wrong audio. The selection is therefore made from the recordings for which timing data exists, and is fixed before the reference audio is bundled.

6. Technology Stack and Justification

Layer	Choice
Recitation analysis	Quran Muaalem model (MIT licence)
Where analysis runs	On a server
Mobile application	Flutter
Backend	FastAPI (Python)
Database	PostgreSQL
Quranic content	Bundled locally, from the King Fahd Complex
Word-level audio timings	Released timing data, verified against the chosen text
Authentication	Managed identity provider

6.1 Recitation analysis — Quran Muaalem model

Alternatives considered: training a model from scratch; a third-party recitation-correction API; forced alignment with hand-written rules.

Training a model was ruled out on feasibility. The authors trained it on an H200 GPU with 141 GB of memory, and the repository publishes no training scripts. That is not reproducible within a graduation project.

A third-party API was ruled out on dependability. The one identified publishes no pricing, no terms of service and no privacy policy, and carries a fault its developers acknowledged in July 2026 that remains unresolved. It would place the project's core feature on a service with no agreement and no recourse, and would send users' voice recordings to a vendor with no published privacy policy.

Hand-written rules over forced alignment were ruled out for two reasons. The best available components carry non-commercial licences — one states the work *"is never for sale, so you may not charge for it, for access to it, or for any feature it powers"* — and the approach would require the team to invent and then defend its own duration thresholds, with no published figures to cite.

The chosen model is openly licensed throughout: the model, its inference library and its base encoder are all MIT. It requires no training run. And it has per-rule accuracy published in a peer-reviewed paper, so the project can cite measured figures rather than assert its own.

A limit that must be stated honestly: the published evaluation covers three tajweed rule families — qalqalah, ghunnah and madd. Other rules either have no published reliability or perform poorly; ikhfa in particular scores close to chance. The project should scope its tajweed claims to what is evidenced.

6.2 Where the analysis runs — on a server

Alternatives considered: running the model on the device; splitting simple checks onto the device and tajweed onto a server.

Running on the device would be better for privacy and would work offline. It was rejected because the standard tooling for converting this model architecture to mobile formats does not support it, which would make the project's core feature depend on unproven engineering. The model is also large, and no published figures exist for its speed on a phone.

Splitting the work was rejected as building two systems instead of one, which is not realistic in a single implementation semester.

6.3 Mobile application — Flutter

Alternatives considered: separate native Android and iOS applications; a web application.

Because the analysis runs on a server, the client's work is screens, microphone capture, a local database and audio playback. Flutter handles all of these, and one codebase producing both platforms roughly halves the client effort for a three-person team.

Separate native applications would give marginally better performance and more direct microphone access, but would double the client work for a benefit the project does not need, and iOS development would additionally require a Mac.

A web application would be easiest to demonstrate, but microphone capture in mobile browsers is less reliable, offline storage of the bundled content is more fragile, and the requirements describe a mobile application.

6.4 Backend — FastAPI in Python

Alternatives considered: Node.js; Django.

The speech model is a Python library. Using Python for the backend avoids a second language boundary between the application logic and the analysis, and lets the same team members work on both. FastAPI was preferred to Django because the backend serves an API and renders no pages, so Django's template and admin facilities would be unused weight.

6.5 Database — PostgreSQL

Alternatives considered: MySQL; a document database.

The data is relational: students, teachers, classes, enrolments, sessions and errors, all connected. Teacher access is scoped by an active enrolment, which is naturally expressed as a join and therefore easy to enforce consistently in one place. A document database would make that access rule harder to enforce correctly, since the check would have to be repeated wherever data is read. MySQL would serve equally well; PostgreSQL was preferred for its stronger handling of structured data types.

6.6 Quranic content — bundled, from the King Fahd Complex

Alternatives considered: fetching from an online Quran service; a different digital edition; serving it from the project's own backend.

Bundling removes any dependency on a third party being available, allows reading and review to work offline, and means a demonstration cannot fail because an external service is down. Juz Amma is small

enough to make this practical — which turns the project's chosen scope into a genuine engineering advantage. The King Fahd Complex edition was chosen for textual authority, as set out in Section 5.

6.7 Word-level audio timings

Alternatives considered: generating the alignments from scratch.

Released timing data was verified to align with the chosen text across all 564 verses of Juz Amma, subject to one documented rule concerning the Basmalah. Because it aligns, no conversion work is needed and generating alignments from scratch would be effort spent for no gain. The timings are specific to a particular recording, so the bundled reference audio must be the recording they were produced from.

6.8 Authentication — a managed identity provider

Alternatives considered: writing a login system; social sign-in only.

Password storage, reset flows and session handling are a concentrated source of security defects, and a failure there would damage the project more than writing it would earn. A managed provider also supports proper session revocation, which matters because withdrawing consent must actually take effect rather than waiting for a token to expire.

Social sign-in alone was rejected because it requires every user to hold an account with a particular company, which is awkward when a teacher enrolls a young student.

To disclose in the privacy section: the managed provider runs from data centres outside the Kingdom, so account identifiers — not recitation audio — cross the border. **Excluded:** phone-number sign-in with SMS codes, common in the region but billed per message, which the project has no budget for.

6.9 What Is Adopted Versus Built

Adopted — must be credited, cannot be claimed	Built by the team — the contribution
The speech model and its base encoder	The Juz Amma reference layer for all 564 verses
The published per-rule accuracy figures	Mapping a detected error back to the exact word on screen
The Quranic text	The comparison and error-classification logic
The word-level timing data	A test set of real learner recitations
The application framework, web framework and database	Per-rule evaluation on that test set
	The feedback design, including when to stay silent

The mapping problem is a genuine contribution rather than glue code. The organisers of a 2026 pronunciation-assessment challenge state that *"there is no trivial or universal mapping from a predicted erroneous phoneme back to the corresponding Arabic character or diacritic."* Solving it for a bounded scope is defensible work.

7. Requirements Mapped to Architecture

7.1 Functional Requirements

Requirement	Component that satisfies it
FR-01 — Create account	Managed identity provider; backend user records
FR-02 — User login	Managed identity provider
FR-03 — Manage profile	Backend user records; Recitation UI
FR-04 — Browse Juz Amma	Bundled Juz Amma Data; Recitation UI
FR-05 — Select surah	Bundled Juz Amma Data; Recitation UI
FR-06 — Display surah verses	Bundled Juz Amma Data (Uthmani text); Recitation UI
FR-07 — Select AI mode	Recitation UI; Session Service
FR-08 — AI-Mujawwid mode	Analysis Service; Comparison Engine
FR-09 — AI-Mushajji mode	Progress Service; Recitation UI
FR-10 — AI-Mu'allim mode	Class Service
FR-11 — Start recitation session	Session Service
FR-12 — Capture recitation audio	Microphone Capture
FR-13 — Analyze recitation	Analysis Service
FR-14 — Detect recitation errors	Analysis Service; Comparison Engine
FR-15 — Identify error location	Comparison Engine; word index derived from the bundled data
FR-16 — Provide recitation feedback	Recitation UI
FR-17 — Provide audio assistance	Reference Audio Player; bundled word timings
FR-18 — Continue recitation	Session Service
FR-19 — End recitation session	Session Service
FR-20 — Display session results	Session Service; Recitation UI
FR-21 — Track user progress	Progress Service
FR-22 — Track practice streaks	Progress Service

7.2 Non-Functional Requirements

Requirement	How the architecture addresses it
NFR-01 — Accuracy	Analysis Service. Measured against a separately collected test set in IS499, not asserted. Tajweed claims scoped to rule families with published reliability.
NFR-02 — Performance	Per-verse segmentation keeps each request small. A pre-warmed analysis service avoids the cold-start delay.
NFR-03 — Real-time interaction	Analysis is per verse rather than per session, so feedback arrives during recitation rather than after it.
NFR-04 — Usability	Recitation UI; feedback stated as an error type on a specific word rather than a raw score.
NFR-05 — Reliability	Content bundled locally, so reading works without a network. The analysis service is isolated, so its failure does not stop the rest of the application.
NFR-06 — Security	Managed identity provider; enrolment-scoped access checked in the data layer; encryption in transit.
NFR-07 — Privacy	Recitation audio never persisted; consent recorded separately per purpose; no voice-identification features.
NFR-08 — Scalability	The analysis service is stateless and can be scaled independently of the backend.
NFR-09 — Maintainability	Analysis, application logic and client are separate, so each can change without the others.
NFR-10 — Compatibility	Flutter client on Android and iOS, using the device microphone.
NFR-11 — Language support	Bundled Arabic Quranic content; interface localization for Arabic and English.

7.3 Notes on Coverage

Every requirement maps to at least one component, and every component serves at least one requirement. This was checked in both directions. Three points are worth drawing out.

FR-15 depends on work described in Section 5. Identifying the location of an error is not something the speech model does — it reports sounds, not positions in a verse. The mapping from a detected sound back to a word on screen is built by the team, and rests on the word-boundary verification.

NFR-01 carries a dependency outside the architecture. The architecture supports measuring accuracy, but the measurement itself depends on a set of learner recordings collected as a separate research activity. That collection is a scheduled IS499 activity in its own right, not a by-product of building the system.

NFR-07 is satisfied by a design decision rather than a mechanism. The privacy requirement is met principally by not retaining recitation audio at all, which is stronger than protecting stored audio well.

8. Design Decisions

8.1 Why recitation audio is not stored

Storing the audio would create the system's most sensitive asset and its heaviest obligations. Not storing it removes both. The per-word results are what the product actually needs — they drive the mastery score, the error history and the teacher view — and they are not voice recordings.

Three considerations supported this. The project is non-commercial and has no reason to accumulate voice data. A substantial share of users are expected to be children, for whom recordings carry particular sensitivity. And the recordings would capture people making mistakes while reciting the Quran, which many would reasonably prefer not to have retained.

The system also does not compute speaker embeddings or any other voice-identification feature. Nothing in Itqan requires it.

A consequence to plan for: evaluating the system's accuracy in IS499 requires recordings of real learners. These must be gathered as a separate, explicitly consented research activity, with participants who know they are being recorded for that purpose. They are not a by-product of ordinary use. The project's ethical approval covers this.

8.2 Where the analysis service runs, and cold start

The analysis model requires a GPU. Two consequences follow.

Cold start. When the service has been idle, the first request must load the model before it can answer, and reference deployments of this model report that first request taking tens of seconds. Every subsequent request is fast. A user waiting to recite will not tolerate this, and during a demonstration it makes a working system look broken. The mitigation is to keep an instance warm for the duration of a session and throughout any demonstration, and to have the application display a clear preparing state rather than appearing frozen.

Where the audio travels. The current arrangement places the application backend on a free cloud tier and the analysis service on a rented GPU service outside Saudi Arabia. Recitation audio, while never stored, therefore travels outside the Kingdom for processing. Saudi Arabia's Personal Data Protection Law regulates such transfers, and the available mechanisms are not practically obtainable by an undergraduate team. The design limits the exposure — nothing is retained anywhere — but the position is stated plainly here rather than left unaddressed.

Hosting the analysis service on departmental hardware inside the Kingdom was considered and remains the stronger option on both counts: the transfer question would not arise, and cold start would be eliminated by keeping the model permanently loaded. It was not adopted for IS498 because it depends on hardware the project does not control. The rented service was chosen for its availability and predictability during development and demonstration.

8.3 Teacher access

A teacher's access is modelled as a scoped, time-bounded link rather than as a role. Authorization depends on the combination of teacher, student, class and an active enrolment period, and is checked on every read rather than enforced only in the interface. When enrolment ends, access ends. A student, or a guardian in the case of a child, can revoke the link, and revocation takes effect immediately.

A teacher sees error types, word locations, mastery figures and trends for their own enrolled students. A teacher does not see the recitation audio, the student's contact details, or activity outside the enrolment period.

How the link is created. The teacher sends an invitation, which the student accepts. For a child, the guardian accepts rather than the child. Enrolment begins when the invitation is accepted, not when it is sent, so an unaccepted invitation grants no access. This direction was chosen because it matches how a halaqah is formed in practice — a teacher gathers students — and because it means access always rests on an act of acceptance by the student or guardian, which is what makes the consent meaningful.

The access rule is covered by two automated tests: a teacher cannot read a student who is not theirs, and a former teacher cannot read a former student.